

CSCE 3301 – Computer Architecture  
Fall 2025  
Project 1: femtoRV32  
RISC-V FPGA Implementation and Testing

Mennatallah Essam  
Seba Wahba  
Sara Hossam Faheem

November 24, 2025

**Abstract**

This report documents the design, implementation, and testing of femtoRV32, a pipelined RISC-V RV32I processor implemented on the Nexys A7 FPGA board. The processor features a 3-stage pipeline with every-other-cycle instruction issuing to handle structural hazards from a unified single-port memory. The implementation includes comprehensive hazard handling through forwarding and stalling mechanisms. The design successfully executes all required RV32I instructions and demonstrates robust performance through extensive testing.

## Contents

|          |                                  |          |
|----------|----------------------------------|----------|
| <b>1</b> | <b>Introduction</b>              | <b>3</b> |
| <b>2</b> | <b>Design Overview</b>           | <b>3</b> |
| 2.1      | Processor Architecture . . . . . | 3        |
| 2.2      | Datapath Components . . . . .    | 3        |
| <b>3</b> | <b>Implementation Details</b>    | <b>4</b> |
| 3.1      | ALU Design . . . . .             | 4        |
| 3.2      | Control Unit . . . . .           | 5        |
| 3.3      | Hazard Handling . . . . .        | 5        |
| 3.3.1    | Forwarding Unit . . . . .        | 5        |
| 3.3.2    | Hazard Detection Unit . . . . .  | 5        |
| 3.4      | Pipeline Registers . . . . .     | 5        |

|          |                                                   |          |
|----------|---------------------------------------------------|----------|
| <b>4</b> | <b>Challenges and Solutions</b>                   | <b>6</b> |
| 4.1      | Memory Structural Hazard . . . . .                | 6        |
| 4.2      | Data Hazard Resolution . . . . .                  | 6        |
| 4.3      | Branch Handling . . . . .                         | 6        |
| 4.4      | Instruction Alignment and PC Management . . . . . | 6        |
| 4.5      | Clock Timing and Pipeline Balancing . . . . .     | 6        |
| 4.6      | Forwarding Unit Complexity . . . . .              | 6        |
| 4.7      | FPGA Resource Limitations . . . . .               | 7        |
| 4.8      | Debugging and Verification . . . . .              | 7        |
| 4.9      | Integration of Multiple Modules . . . . .         | 7        |
| 4.10     | System Performance Optimization . . . . .         | 7        |
| <b>5</b> | <b>Testing Methodology</b>                        | <b>7</b> |
| <b>6</b> | <b>Results and Performance</b>                    | <b>7</b> |
| <b>7</b> | <b>Conclusion</b>                                 | <b>8</b> |

# 1 Introduction

The femtoRV32 project implements a complete RISC-V RV32I processor core following the project specifications. The processor supports 37 user-level instructions from the RV32I base instruction set, with ECALL, EBREAK, PAUSE, FENCE, and FENCE.TSO instructions implemented as halt operations.

Key features of our implementation include:

- 3-stage pipelined architecture with hazard handling
- Single unified memory for instructions and data
- Every-other-cycle instruction issuing ( $\text{CPI} = 2$ )
- Comprehensive forwarding and stalling mechanisms
- FPGA implementation on Nexys A7 board

## 2 Design Overview

### 2.1 Processor Architecture

The femtoRV32 employs a modified 3-stage pipeline structure that spans 6 clock cycles:

- **Stage 0 (2 cycles):** Instruction Fetch (C0) + Register Read (C1)
- **Stage 1 (2 cycles):** ALU Operation (C0) + Memory Access (C1)
- **Stage 2 (1 cycle):** Register Write Back (C0)

This design effectively handles the structural hazard introduced by the single-port memory while maintaining pipeline efficiency.

### 2.2 Datapath Components

The processor datapath consists of the following key components:

- Program Counter (PC) and instruction fetch logic
- 32-entry register file with dual read ports and single write port
- Arithmetic Logic Unit (ALU) supporting all RV32I operations
- Immediate generator for instruction decoding
- Control unit for instruction interpretation
- Hazard detection and forwarding units

- Unified instruction/data memory



Figure 1: femtoRV32 Pipelined Datapath

## 3 Implementation Details

### 3.1 ALU Design

The ALU implements all required RV32I arithmetic, logical, and comparison operations. Key features include:

```
1 module ALU#(parameter n=32)(  
2     input [n-1:0] a,  
3     input [n-1:0] b,  
4     input [3:0] sel,
```

```

5     output reg[n-1:0] c,
6     output wire zeroflag
7 );

```

Listing 1: ALU Module Interface

The ALU supports ADD, SUB, AND, OR, XOR, SLT, SLTU, SLL, SRL, and SRA operations.

## 3.2 Control Unit

The control unit decodes instructions and generates control signals:

```

1 module control(
2     input [6:0] opcode,
3     output reg branch, output reg MemRead, output reg MemToReg,
4     output reg [1:0] ALUOp, output reg MemWrite, output reg ALUSel,
5     output reg RegWrite
6 );

```

Listing 2: Control Unit Implementation

## 3.3 Hazard Handling

### 3.3.1 Forwarding Unit

```

1 module forwardingUnit(
2     input EX_MEM_RegWrite, input [4:0] EX_MEM_RD,
3     input MEM_WB_RegWrite, input [4:0] MEM_WB_RD,
4     input [4:0] ID_EX_rs1, input [4:0] ID_EX_rs2,
5     output reg [1:0] forwardA, output reg [1:0] forwardB
6 );

```

Listing 3: Forwarding Unit Implementation

### 3.3.2 Hazard Detection Unit

```

1 module HazardDetectionUnit(
2     input [4:0] IF_ID_rs1, input [4:0] IF_ID_rs2,
3     input [4:0] ID_EX_RD, input ID_EX_MemRead,
4     output reg stall
5 );

```

Listing 4: Hazard Detection Unit

## 3.4 Pipeline Registers

```

1 module IF_ID_Register(
2     input clk, input rst, input stall, input flush,
3     input [31:0] instruction_in, input [31:0] PC_plus_4_in,
4     output reg [31:0] instruction_out, output reg [31:0] PC_plus_4_out

```

## 4 Challenges and Solutions

### 4.1 Memory Structural Hazard

**Challenge:** The single-port memory could cause instruction fetch and data access conflicts, stalling the pipeline.

**Solution:** Implemented every-other-cycle instruction issuing, creating a 6-cycle 3-stage pipeline and avoiding memory conflicts.

### 4.2 Data Hazard Resolution

**Challenge:** Read-after-Write (RAW) hazards could lead to incorrect computations.

**Solution:** Developed forwarding logic for EX and MEM stages, and hazard detection for load-use stalls that cannot be resolved with forwarding.

### 4.3 Branch Handling

**Challenge:** Branch instructions introduce control hazards.

**Solution:** Branch resolved in EX stage; pipeline flushes instructions on branch taken. This guarantees correct execution.

### 4.4 Instruction Alignment and PC Management

**Challenge:** 2-cycle fetch stage complicates PC updates and instruction alignment.

**Solution:** Carefully designed PC logic and flush mechanisms for branch misprediction.

### 4.5 Clock Timing and Pipeline Balancing

**Challenge:** Ensuring all stages complete within clock cycle limits due to varying delays.

**Solution:** Split operations across 2-cycle stages and validated timing with FPGA simulation.

### 4.6 Forwarding Unit Complexity

**Challenge:** Correctly forwarding the newest data without introducing hazards.

**Solution:** Two-level priority logic: EX stage forwarding  $\wedge$  MEM stage forwarding. Verified using directed tests.

## 4.7 FPGA Resource Limitations

**Challenge:** Limited LUTs and registers on Nexys A7 board.

**Solution:** Optimized pipeline registers and control signals. Efficiently reused ALU and memory ports.

## 4.8 Debugging and Verification

**Challenge:** Observing internal pipeline signals on FPGA is hard.

**Solution:** Developed extensive test benches, signal probes, and waveform analysis.

## 4.9 Integration of Multiple Modules

**Challenge:** Connecting ALU, control unit, forwarding, hazard detection, and memory caused timing issues.

**Solution:** Modular development with incremental integration and verification at each step.

## 4.10 System Performance Optimization

**Challenge:** Achieving CPI of 2 while handling hazards.

**Solution:** Carefully designed pipeline, forwarding, and every-other-cycle instruction issue; verified using test programs.

# 5 Testing Methodology

We verified:

- All instruction types
- Data hazards and forwarding
- Control hazards
- Memory operations

# 6 Results and Performance

The processor supports all 37 RV32I instructions and achieves  $\text{CPI} = 2$ .

| Feature             | Target | Achieved |
|---------------------|--------|----------|
| Instruction Support | 37     | 37       |
| Pipeline Stages     | 3      | 3        |
| CPI                 | 2      | 2        |
| Hazard Handling     | Full   | Full     |

Table 1: Performance Summary

## 7 Conclusion

The femtoRV32 processor is fully functional and meets all project requirements. It provides a solid platform for learning RISC-V design and FPGA implementation.

## Appendix

### A. File Structure

```
project/
  README.txt
  journals/
    mennatallah_journal.txt
    seba_journal.txt
    sara_journal.txt
  verilog/
    defines.vh
    ALU.v
    alucontrol.v
    control.v
    DFlipFlop.v
    forwardingUnit.v
    HazardDetectionUnit.v
    IMem.v
    femtorv32.v
  test/
    basic_tests/
    hazard_tests/
    compliance_tests/
```